

Chapitre 4: Persistance de données en PHP (PDO)



ING3

2025/2026

S.NOUFEL



Plan

- Introduction
- Type d'interaction avec la BD
- Librairie MYSQLI
- Présentation du PHP Data Object - PDO
- Cas de requêtes de Sélection
- Cas de requêtes de mise à jour
- Sécurité d'exécution des requêtes

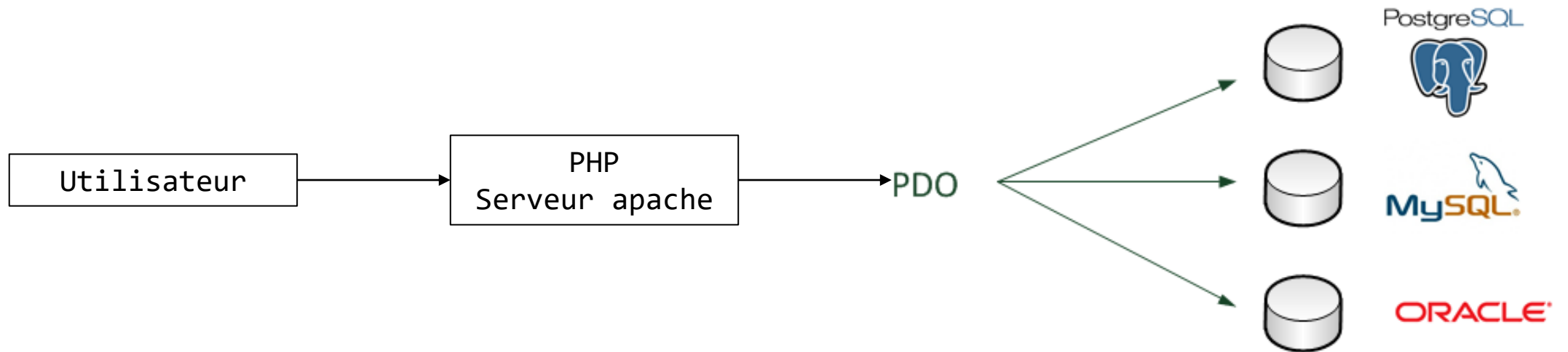
Introduction

- PHP permet d'assurer la persistance des données
- PHP Data Object est une interface pour manipuler une base de données à partir du PHP
- PDO support plusieurs SGBD (MySQL, PostgreSQL, Oracle, SQLite etc.)
- PHP-PDO support par défaut les BD MYSQL
- PDO est plus sécurisé

Il existe d'autre interface pour la manipulation des bases de données à partir de PHP

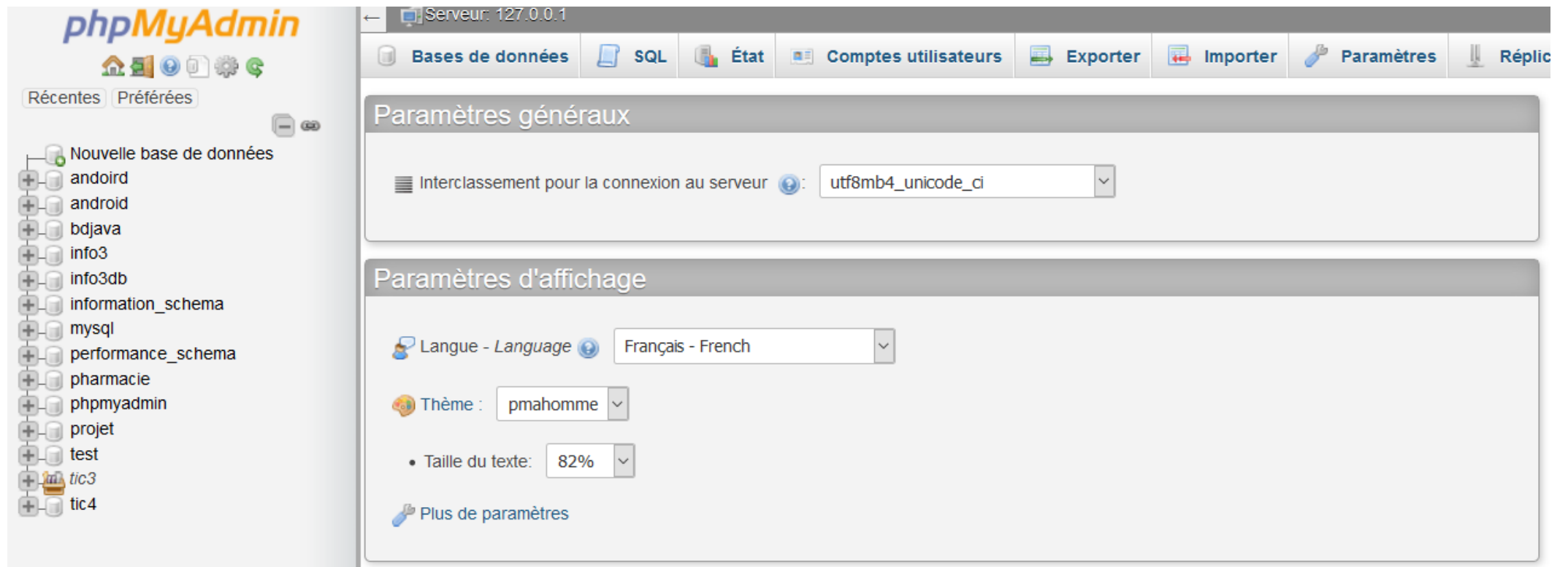
Introduction

- Le rôle des extensions de PHP pour l'interaction avec la BD
- Ces extensions permettent de gérer l'hétérogénéité des SGBD



phpMyadmin

- Votre serveur d'application (xampp/ wamp) intègre par défaut les outils pour administrer vos bases de données



phpMyAdmin

- Facilite la création des BD
- Fournit un outil graphique pour la création des tables
- Facilite la manipulation des tables et des données

Bases de données

Création d'une base de données : ?

etudiantBD

Interclassement

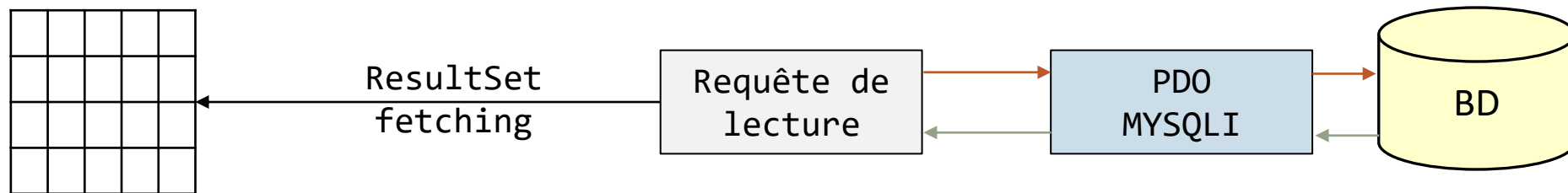
Créer

Nom	Type ?	Taille/Valeurs* ?	Valeur par défaut ?	Interclassement	Attributs	Null	Index
idEtd Choisir à partir des colonnes centrales	INT		Aucun(e)			☐	PRIMARY
nomEtd Choisir à partir des colonnes centrales	VARCHAR	40	Aucun(e)			☐	---
MoyenneEtd Choisir à partir des colonnes centrales	FLOAT		Aucun(e)			☐	---

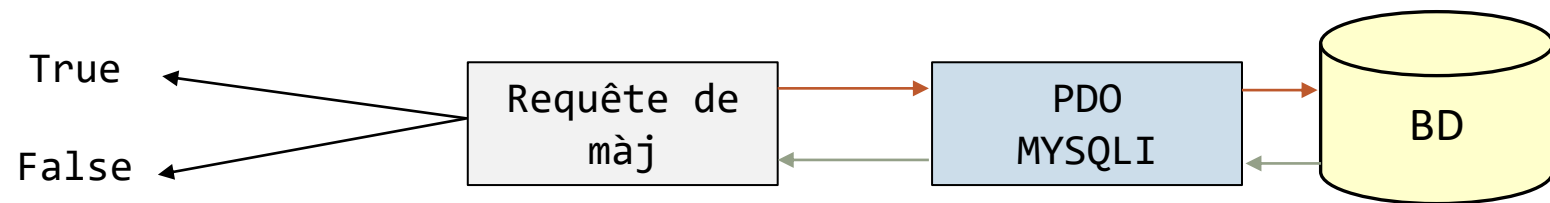
Interaction avec la BD

Les interactions avec la base de données sont structurées ont deux types:

1) Requêtes de lecture



2) Requêtes de mise à jour



La librairie MYSQLI

Les fonctions de MYSQLI

- **New mysqli(serveur, user, password, BD):** permet la création d'un objet de connexion mysqli pour l'exécution des objets
- **objetCnx->query():** exécuter une requête SQL
- **objetCnx->connect_error():** vérifier les erreurs de connexion
- **resultSet->fetch_assoc():** récupérer les résultats d'une requête sous forme de tableau associatif

La librairie MYSQLI

- Nous supposons que la base de données « **etudiantBD** » existe
- Cette base de données contient la table « **etudiant** »

	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action
☐	1	idEtd 	int(11)			Non	Aucun(e)		AUTO_INCREMENT	 Modifier  Supprimer  Primaire
☐	2	nomEtd	varchar(40)	latin1_swedish_ci		Non	Aucun(e)			 Modifier  Supprimer  Primaire
☐	3	MoyenneEtd	float			Non	Aucun(e)			 Modifier  Supprimer  Primaire

La librairie MYSQLI

La librairie MYSQLI permet de faire la manipulation des BD par l'exécution direct des requêtes

Exemple:

```
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "etudiantbd";
//établir la connexion avec la BD
$conn = new mysqli($servername, $username, $password, $dbname);

// test de l'objet de connexion
if ($conn->connect_error) {
    die("problème de connexion: " . $conn->connect_error); }
```

- Fonction mysqli permet d'établir avec une base de données MySQL
- Nécessite quatre paramètres: @adresse du serveur, userName, mot de passe, nom de la BD

La librairie MYSQLI

- MYSQLI permet une exécution directe des requêtes SQL
- La méthode query permet d'exécuter le code SQL
- Exemple d'une requête de mise à jour

```
$nom = "saad";  
$moyenne = 12;  
$sql = "INSERT INTO etudiant VALUES ( 2, '$nom', $moyenne)";  
  
// exécution de la requête  
if ($conn->query($sql) === TRUE) {  
    echo "Etudiant ajouter avec succes";  
} else {  
    echo "Erreur: " . $sql . "<br>" . $conn->error;  
}
```

La librairie MYSQLI

Préparation de la requête Insert dans mysqli :

```
$sql = $conn->prepare("INSERT INTO etudiant (id, nom, moyenne) VALUES (?, ?, ?)");  
$sql->bind_param("isi", $id, $nom, $moyenne);  
  
// Exécution  
if ($sql->execute()) {  
    echo "Étudiant ajouté avec succès";  
} else {  
    echo "Erreur: " . $sql->error;  
}
```

La librairie MYSQLI

- La librairie d'exécution des requêtes de sélection de données
- Exemple de requête de sélection

```
// établir la connexion
$conn = new mysqli("localhost", "root", "", "etudiantbd");

// execution de la requête
$result = $conn->query("SELECT * FROM etudiant");

// fetching des résultats
while ($row = $result->fetch_assoc()){
    echo htmlentities($row['nomEtd']) . "- " . $row['moyenneEtd'] . "<br/>";
}
```

La librairie PDO

La librairie PDO :

- Est la librairie la plus récente pour la manipulation d'une base de données;
- Assure un niveau de sécurité plus élevé
- Protège contre les injections SQL

La librairie PDO

Les étapes détaillées pour l'exécution d'une requête en PDO sont:

- Établir la connexion (objet de connexion PDO)
- Créer la requête SQL
- Créer la requête préparée
- Binding des paramètres
- Exécution de la requête préparée

La librairie PDO (étape 1- connexion PDO)

- La connexion avec la base de données passe l'instanciation d'un objet de connexion
- De la même façon que mysqli, il faut spécifier l'adresse du serveur de la BD, le nom de la base de données, le login de l'utilisateur et son password.

Exemple:

```
$pdo = new PDO('mysql:host=localhost;dbname=etudiantbd', 'root', '');
```

- L'objet **\$pdo** représente la base pour l'exécution des requêtes SQL
- L'objet **\$pdo** permet la préparation de la structure des requêtes (**plus de sécurité**)

La librairie PDO (étape 2- création de la requête sql)

- PDO permet la création de la structure de la requête SQL à exécuter
- La requête SQL est enregistrée dans une variable de type String
- Par exemple:

```
// requête sql  
$requete = 'INSERT INTO ETUDIANT VALUES (?, ?, ?)';
```

- « ? » : définit l'emplacement des paramètres à passer à la requête
- Chaque « ? » Sera remplacé par une valeur

La librairie PDO (étape 3- création de la requête préparée)

- La création de la requête préparée passe par l'utilisation de la méthode « prepare »
- Cette méthode est une méthode déclenchée à partir d'un objet de connexion PDO

```
$objetPDO->prepare($requeteSQL)
```

```
// preparation de la requete  
$requetePrep = $pdo->prepare($requeteSQL);
```

La librairie PDO (étape 4- Binding des valeurs)

- La méthode `bindValue` permet d'assigner des paramètres dans les emplacements des différents « ? » de la requête SQL.
- La méthode `bindValue` reçoit en paramètre la position du paramètre dans la requête, la valeur à assigner, et le type du paramètre.
- Syntaxe:

```
$requetePreparé->bindValue($position, $value, $type)
```

- Exemple:

```
$requetePrep->bindValue(1, $this->idEtd , PDO::PARAM_INT);  
$requetePrep->bindValue(1, $this->nomEtd , PDO::PARAM_STR);  
$requetePrep->bindValue(2, $this->moyenneEtd , PDO::PARAM_STR);
```

La librairie PDO (étape 4- Binding des valeurs)

Les types de paramètres sont définis dans la classe PDO

Les types supportés en PDO:

- PDO::PARAM_INT: pour les entiers
- PDO::PARAM_STR: pour les chaînes de caractère
- PDO::PARAM_BOOL: pour les booléens

On utilise le type **PDO::PARAM_STR** pour les autres types de paramètres (par exemple les paramètres de type date, les floats)

La librairie PDO (étape 5- Exécution de la requête)

Il existe trois fonctions pour l'exécution d'une requête en PDO

- Fonction `exec()`: retourne le nombre de lignes affectées par la requête. Elle ne revoit pas de résultat pour la requête `select`.
- Fonction `query()` : permet la récupération des résultats d'une requête de selection
- Fonction `execute()`: permet de réutiliser la requête et de récupérer les résultats de `SELECT`.

```
$resultat = $requetePrep->execute();
```

Exercice d'application

- Créer les fonctions de base CRUD pour la manipulation de la table des étudiants en utilisant la lib PDO.
- Regrouper toutes les fonctionnalités dans une seule application.

Rappel:

	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra
☐	1	idEtd 	int(11)			Non	Aucun(e)		AUTO_INCREMENT
☐	2	nomEtd	varchar(40) latin1_swedish_ci			Non	Aucun(e)		
☐	3	MoyenneEtd	float			Non	Aucun(e)		

Exemple de requête de mäj

Les méthodes du CRUD sont déclarées en tant que méthode d'instance de la classe suivante

```
class Etudiant{  
    public $idEtd;  
    public $nomEtd;  
    public $moyenneEtd;  
  
    public function __construct($idEtd, $nomEtd, $moyenneEtd){  
        $this->idEtd = $idEtd ;  
        $this->nomEtd = $nomEtd ;  
        $this->moyenneEtd = $moyenneEtd;  
    }  
}
```

Exemple de requête de mäj

```
public function insertEtd(){
    $resultat = -1;
    try{
        // objet de connexion
        $pdo = new PDO('mysql:host=localhost;dbname=etudiantbd', 'root', '');
        // requete sql
        $requete = 'INSERT INTO ETUDIANT VALUES (NULL, ?, ?)';
        // preparation de la requete
        $requetePrep = $pdo->prepare($requete);
        // binding paramètres valeurs
        $requetePrep->bindValue(1, $this->nomEtd , PDO::PARAM_STR);
        $requetePrep->bindValue(2, $this->moyenneEtd , PDO::PARAM_STR);
        // execution de la requête
        $resultat = $requetePrep->execute();
    }catch (PDOException $e ){
        echo "probleme d'ajout de l'etudiant " . $e->getMessage();
        $resultat = false;
    }
    return $resultat;
}
```


Exemple de requête de lecture

Méthode pour afficher tous les étudiants

```
public function lister(){  
    $pdo = new PDO('mysql:host=localhost;dbname=etudiantbd', 'root', '');  
    $req = 'SELECT * FROM ETUDIANT';  
    $resultSet = $pdo->query($req);  
  
    while($ligne = $resultSet->fetch() ) {  
        echo $ligne['idEtd'] . " - " . $ligne['nomEtd'] . "<br/>";  
    }  
}
```

Exercice d'application

Soit la classe Produit suivante

Produit	
- idProd	: int
- designationProd	: String
- qteProd	: int
- puProd	: java.lang.Float

➤ Proposer le code nécessaire pour l'implémentation des opérations CRUD de cette classe.